Unstructured Mesth with CF mesh
Technical Progress Report (Phase 4)

Pranjal Kumar Shukla

FOSSEE Internship – Progress Report

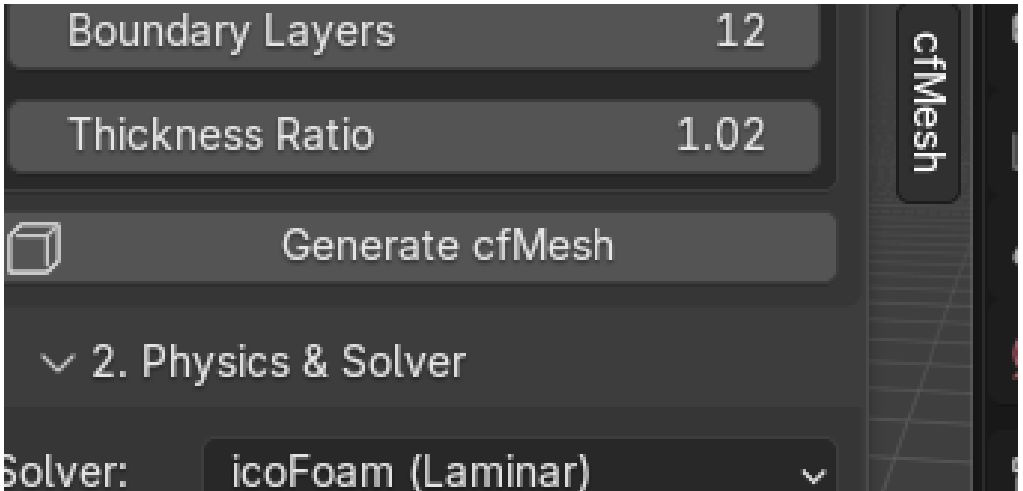
SCAI School

VIT Bhopal University

March 28, 2026

1. Turbulence Modeling (RANS)

- **Objective**: Graduate from simple laminar flow to industrial-grade turbulence models as planned in Week 3.
- **Implementation**:
  - Added UI Enum Property for `kEpsilon` and `kOmegaSST` selection.
  - The Python backend now automatically provisions `constant/turbulenceProperties`.
  - Dynamically injects boundary initialization files (`0/k`, `0/epsilon`, `0/omega`, `0/nut`) utilizing standardized OpenFOAM `WallFunctions`.



2. Codebase Modularization

- **Phase 3 Limitation**: The entire add-on logic was contained within a single colossal `run_cfmesh.py` file (1000+ lines).
- **Phase 4 Refactor**: Split the monolithic codebase into highly specialized Python modules:
  - `ui.py`: Sidebar drawing logic and panel layouts.
  - `properties.py`: Global states and Blender Scene properties.
  - `operators.py`: Execution callbacks and async threads.
  - `utils_mesh.py`: Pure OpenFOAM dictionary string generation.
  - `utils_system.py`: Safe subprocess execution and OS-level I/O.

3. Pre-Execution Input Validation

Silent Failures

Previously, invalid directories or meshing empty geometry would silently crash OpenFOAM without telling the Blender user why.

The Solution (Mega Exception Handling):

- We implemented programmatic "Guards" before *any* files are written.
- Checks array lengths: If the selected object has 0 polygons, an explicit Blender Popup catches the error and halts execution.
- Validates writable OS permissions on the Export Directory.
- Warns the user of excessive Boundary Layers (> 10) using native `self.report` tools.

4. Real-time OpenFOAM Stderr Parsing

- We updated the Async Subprocess to intercept the raw terminal streams from OpenFOAM.
- If `icoFoam` or `cartesianMesh` throws a `FATAL ERROR`, the exact cause (e.g., Matrix Divergence, Courant Number exceeded) is caught and printed directly into a **Red Alert Box** in the Blender N-Panel.

5. Tracking Week 4 Roadblocks

Extensive refactoring led to several hard-earned lessons:

- **Blender 5.1 Extension Strictness**: The modern `blender_manifest.toml` structure entirely rejected our old ZIP packaging format. We learned to package directly from the root and provide dummy `website` strings to satisfy validation.
- **Regression Tracking**: Extracting modules accidentally overwrote our core `cartesianMesh` engine with `snappyHexMesh`. It reinforced the importance of aggressively checking core objectives against refactored results.
- **STDOUT vs STDERR Handling**: OpenFOAM logs `FATAL ERROR` traces to Standard **Output**, not Standard **Error**. Our Python interceptor had to be rewritten to merge both OS streams to successfully catch CLI crashes.

Thank You!

End of Week 4 Report